MACHINE LEARNING · ENSEMBLE METHODS

Bagging (Bootstrap Aggregating)

A complete visual guide — from the core intuition to the math, the code, and the tuning tricks used in production. Basic to advanced, explained from scratch.

Bias–Variance Tradeoff | Bootstrap Sampling | Classification & Regression | Out-of-Bag Score | scikit-learn Code | Random Forest Preview

Compiled tutorial · Dark-mode edition · 16 sections

CONTENTS

What You'll Learn

This guide is built to be read top to bottom — each section leans on the one before it. If you already know the basics, jump straight to the math (06, 09) or the code (12).

01 **Why Do We Need Bagging?**
The core problem: a single model that is either too simple or too shaky.

02 **The Core Idea**
Bootstrap + Aggregating — the two words hiding inside the name.

03 **Bootstrap Sampling, With Math**
Sampling with replacement, and the probability behind it.

04 **The Aggregation Step**
How votes and averages combine many weak opinions into one strong one.

05 **Step-by-Step Worked Example**
A tiny 10-row dataset, 3 trees, one prediction — done by hand.

06 **Why Bagging Actually Works**
Bias, variance, and the math of why averaging reduces variance.

07 **Seeing It: Decision Boundaries**
Single tree vs. bagged ensemble, plotted side-by-side.

08 **The Four Flavors of Bagging**
Bagging, Pasting, Random Subspaces, Random Patches.

09 **Out-of-Bag (OOB) Score**
Free validation data hiding inside the bootstrap, with the 1/e proof.

10 **Key Hyperparameters**
Every knob you can turn, in one cheat-sheet table.

11 **How Many Trees Do You Need?**
The diminishing-returns curve for n_estimators.

12 **Hands-On: Python From Scratch**
Full BaggingClassifier walkthrough with real code.

13 **Practical Tuning Tips**
Rules of thumb from real experiments across many datasets.

14 **Finding Hyperparameters with GridSearchCV**
Letting search do the guessing for you.

15 **From Bagging to Random Forest**
The natural next step — advanced preview.

16 **Summary Cheat-Sheet**
Everything on one page for quick revision.

SECTION 01

Why Do We Need Bagging?

Every predictive model has to walk a tightrope between two failure modes: being **too simple** to capture the pattern, and being **too sensitive** to the exact data it happened to see.

Imagine you train a **Decision Tree** and let it grow until every leaf is pure — 100% accuracy on training data. That sounds great, until you test it on new data and the accuracy collapses. The tree didn't learn the underlying pattern; it memorised the noise in your specific training set. This is called **overfitting**, and it is a symptom of **high variance**: if you gave the same algorithm a slightly different training set, it would produce a wildly different model.

WARNING · THE CORE PROBLEM

Low-bias, high-variance models (like fully-grown decision trees, KNN with small k) fit the training data beautifully but are unstable — small changes in the data lead to large changes in predictions. We want a model that is both accurate *and* stable.

Bias vs. Variance, in plain words

High Bias	**Model is too simple — underfits, misses real patterns**
High Variance	**Model is too flexible — overfits, memorises noise**
Goal	**Low bias AND low variance (rarely happens naturally)**

Unfortunately, bias and variance usually trade off against each other: push a model to fit the training data harder (lower bias) and it usually becomes shakier on new data (higher variance) — and vice versa. This is the classic **bias–variance tradeoff**, and it's the single biggest reason ensemble techniques like Bagging exist.

KEY IDEA · THE BAGGING INSIGHT

What if, instead of trusting one shaky model, we trained *many* shaky models — each on a slightly different slice of the data — and then let them vote? Individually each model is still unstable, but their **average opinion** turns out to be remarkably stable. That is the entire idea behind Bagging.

This technique was introduced by **Leo Breiman in 1996** and the name is a contraction of its two steps: **B**ootstrap **Agg**regat**ing** → **Bagging**. It is one of the foundational ideas in ensemble learning,

and it directly powers one of the most widely used algorithms in practice: the **Random Forest** (which we'll connect back to in Section 15).

SECTION 02

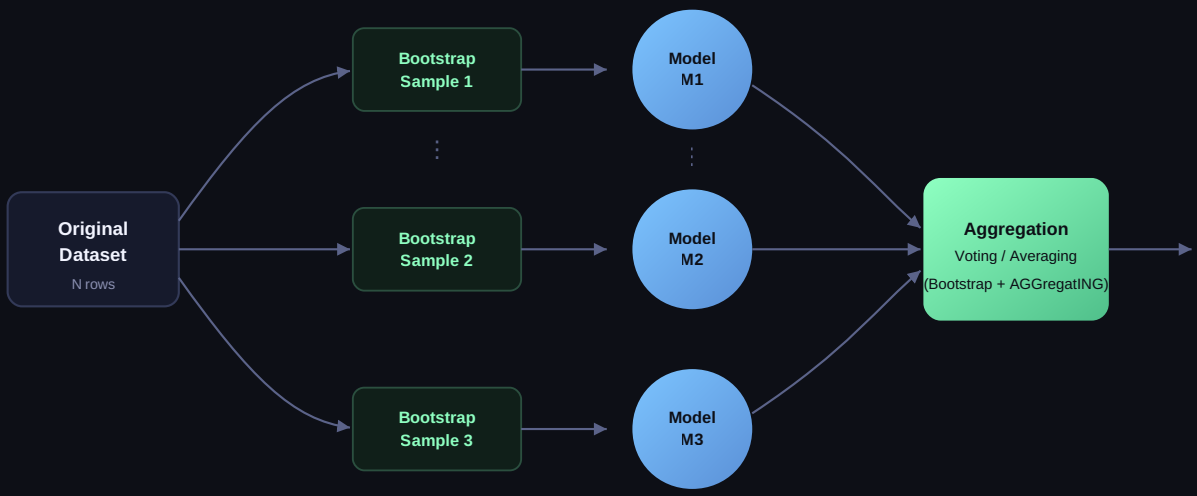
The Core Idea: Bootstrap + Aggregating

Bagging has exactly two moving parts. Understand these two, and you understand the whole algorithm.

1. **1. Bootstrap — create many random sub-datasets**
   From your original training data, draw several random samples (usually the same size as the original dataset), **with replacement**. Each sample is called a "bootstrap sample," and because of replacement, some rows appear multiple times while others don't appear at all.

2. **2. Train a base model on each sample**
   Every bootstrap sample is handed to an identical, freshly-initialised base algorithm (commonly a Decision Tree). Each one trains independently and sees a slightly different version of reality.

3. **3. Aggregate — combine the predictions**
   For classification: take a **majority vote** across all base models. For regression: take the **average** of all predictions. This combined answer is the final output of the Bagging ensemble.



The full Bagging pipeline: one dataset → many bootstrap samples → many base models → one aggregated prediction.

Why does training on "different" data help?

The key requirement for Bagging to work is that the base models must be **different from each other**. If all your base models saw the exact same data, you'd just be training the same model repeatedly — pointless. By feeding each model a different random subset, we deliberately introduce variety. That

variety is later cancelled out by averaging, exactly the way random noise cancels out in scientific measurements when you take many readings instead of one.

ANALOGY · A SIMPLE ANALOGY

Imagine asking one random person to guess the number of jellybeans in a jar — their guess could be way off. Now ask 500 random people and average their guesses. The average tends to land astonishingly close to the true number, even though each individual guess was noisy. This is the "wisdom of the crowd," and Bagging engineers this exact effect for machine learning models.

SECTION 03

Bootstrap Sampling — With the Math

"Bootstrap sampling" simply means: randomly draw rows from your dataset, **one at a time, and put each row back before drawing the next one.** This is called **sampling with replacement**.



A dataset of 10 rows (top) and one bootstrap sample drawn from it (bottom). Because we sample with replacement, row "2" got picked three times, while rows "3", "6", and "8" never got picked at all.

Why "with replacement" instead of just shuffling?

If you sampled *without* replacement and took all N rows, you'd just get back the exact same dataset in a different order — zero variety. Replacement is what creates the differences between bootstrap samples: some rows get emphasised (appear multiple times), others get ignored entirely. That's the randomness Bagging depends on.

The probability a row gets left out

This is one of the most elegant pieces of math behind Bagging. Suppose your dataset has **N** rows, and you draw a bootstrap sample of size N (the usual default).

$$P(\text{a specific row is } \textit{not} \text{ picked in one draw}) = 1 - 1/N$$

SINGLE DRAW

Since we draw N times independently (with replacement), the probability that a specific row is *never* picked in any of the N draws is:

$$P(\text{row never picked}) = (1 - 1/N)^N$$

N INDEPENDENT DRAWS

As N grows large, this expression converges to a beautiful constant:

$$\lim_{N \to \infty} (1 - 1/N)^N = 1/e \approx \mathbf{0.368}$$

E ≈ 2.71828, EULER'S NUMBER

MATH · WHAT THIS MEANS IN PRACTICE

Roughly **36.8%** of the original rows are left out of any given bootstrap sample — no matter how big your dataset is. The remaining **~63.2%** of rows make it into the sample (many of them duplicated). This 63.2% / 36.8% split is not a coincidence or a rough guess — it is a mathematical constant. We'll use the "left-out" 36.8% again in Section 09, where it becomes a free validation set for every base model (the **Out-of-Bag** score).

SECTION 04

The Aggregation Step

Once every base model has been trained on its own bootstrap sample, we need to combine their individual answers into one final answer. How we combine depends on the type of problem.

Classification → Majority Voting

Each base model outputs a class label for a given input. Bagging simply counts the votes and picks the class with the most votes — like a democratic election among models.

EXAMPLE · WORKED MINI-EXAMPLE

Three trees predict the class of the same new point:

Model M1 → class **2** | Model M2 → class **2** | Model M3 → class **3**

Majority vote: class **2** wins (2 votes vs 1). That becomes the ensemble's final prediction, even though one model disagreed.

$$\hat{y} = \text{mode}\ \{\ h_1(x), h_2(x), \ldots, h_B(x)\ \}$$

Ŷ = FINAL CLASS · H_I(X) = PREDICTION OF BASE MODEL I · B = NUMBER OF BASE MODELS

Regression → Averaging

When predicting a continuous number (like a house price), voting doesn't make sense — so Bagging simply averages the predictions of all base models.

$$\hat{y} = (1 / B) \cdot \Sigma_{i=1}^{B}\ h_i(x)$$

SIMPLE MEAN OF ALL BASE-MODEL OUTPUTS

Some implementations use a weighted average (giving more trustworthy models more say), but the plain unweighted average is the standard and by far the most common approach.

KEY IDEA · WHY AVERAGING CANCELS OUT NOISE

Each base model's error can be thought of as "true signal + random noise." If the noise from different models is uncorrelated (which bootstrap sampling encourages), then averaging B models divides the variance of that noise by roughly a factor of B — without touching the signal. This is the mathematical heart of why Bagging reduces variance. We formalise this in Section 06.

SECTION 05

Step-by-Step Worked Example

Let's run the whole algorithm by hand on a tiny toy dataset, exactly the way you'd trace through it on paper before trusting the library to do it for you.

1. **Setup**

   Say we have a training set of 10 rows. We decide to train **3 base decision trees** (in real projects this is usually 100–500, but 3 is enough to see the mechanics clearly), and each tree gets a bootstrap sample of size 10 (same as the original, drawn with replacement).

2. **Draw Bootstrap Sample 1**

   Randomly draw 10 rows with replacement, e.g. rows `[2, 2, 5, 0, 9, 9, 4, 2, 7, 1]`. Train **Tree 1** on exactly this data. It never sees rows 3, 6, 8.

3. **Draw Bootstrap Sample 2**

   Draw again, independently, e.g. `[3, 7, 4, 7, 9, 8, 1, 0, 3, 6]`. Train **Tree 2** on this different mix. It never sees rows 2, 5.

4. **Draw Bootstrap Sample 3**

   Draw a third time, e.g. `[9, 7, 1, 4, 4, 2, 0, 6, 8, 5]`. Train **Tree 3** on this data. It never sees row 3.

5. **Get a new point to classify**

   A brand-new data point X (not seen during training) arrives, and we ask all 3 trees for their opinion.

6. **Collect predictions and vote**

   Tree 1 says class **2**. Tree 2 says class **2**. Tree 3 says class **1**. Majority vote → the ensemble outputs class **2**.

KEY IDEA · WHAT JUST HAPPENED

Even though Tree 3 disagreed, the ensemble's final answer is more trustworthy than any single tree's answer, because it's backed by agreement across differently-trained models. If Tree 1 happened to overfit a quirk in its bootstrap sample, Tree 2 and Tree 3 — trained on different data — are unlikely to share that exact same quirk, so the vote washes it out.

This is exactly what a library call like `BaggingClassifier(n_estimators=3)` does internally — just automated, and usually with far more than 3 trees. We'll see the real code in Section 12.

SECTION 06

Why Bagging Actually Works

This is the "advanced" part most tutorials skip. Let's actually derive why averaging reduces variance.

Decomposing the error

For a model h(x) trying to predict true value y, the expected squared error can be broken into three parts:

$$E[(y - h(x))^2] = \text{Bias}(h(x))^2 + \text{Var}(h(x)) + \sigma^2$$

Σ² = IRREDUCIBLE NOISE IN THE DATA ITSELF

A single, fully-grown decision tree has very low bias (it can fit almost anything) but high variance (its shape changes drastically with the training data). We can't reduce the irreducible noise σ², but we *can* attack the variance term — and that's exactly what Bagging targets, while barely touching the bias term.

The variance-reduction formula

Suppose we average B base models, and each has the same individual variance σ^2_m. If their errors were completely **independent**, the variance of the average would be:

$$\text{Var}(\text{average of B models}) = \sigma^2_m / B$$

VARIANCE SHRINKS LINEARLY WITH MORE MODELS — IF FULLY INDEPENDENT

In reality, bootstrap samples overlap (they're all drawn from the same original dataset), so the base models' errors are **correlated**, not fully independent. With average pairwise correlation ρ between models, the real formula is:

$$\text{Var}(\text{average}) = \rho \cdot \sigma^2_m + [(1-\rho)/B] \cdot \sigma^2_m$$

P = CORRELATION BETWEEN BASE MODELS' ERRORS

MATH · READING THIS FORMULA

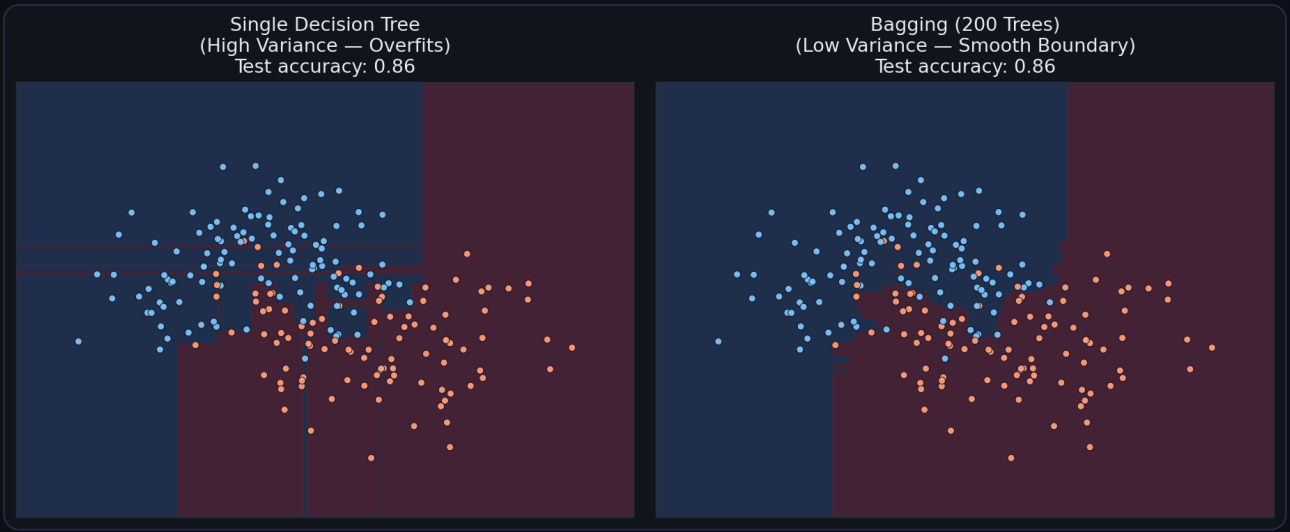
As B (number of trees) grows large, the second term shrinks toward zero — but the first term, $\rho \cdot \sigma^2_m$, **never disappears**. This is precisely why adding more and more trees eventually plateaus (see Section 11's plot): you can drive down the "independent" part of the variance, but the correlated part is a hard floor. This is also exactly why **Random Forest** adds column sampling on top of bagging — it deliberately reduces ρ (the correlation between trees) by forcing trees to look at different features, pushing that floor even lower.

The practical takeaway: bagging trades a tiny bit of bias (averaging slightly smooths the decision boundary) for a large reduction in variance — and since variance is usually the dominant error source for deep trees, the trade is almost always a big net win.

SECTION 07

Seeing It: Decision Boundaries in Practice

Theory is convincing, but seeing it happen on real data makes it click. Below, both models are trained on the exact same noisy, curved ("moons") dataset.



Left: a single decision tree carves out a jagged, overfit boundary with small "islands" chasing individual noisy points. Right: a Bagging ensemble of 200 trees produces a visibly smoother boundary that follows the true shape of the data rather than its noise.

Notice the single tree's boundary has sharp, angular corners and small isolated regions — each one is the tree contorting itself to classify one or two noisy training points correctly. Those contortions don't generalise; they're overfitting in visual form. The bagged ensemble's boundary is far smoother because those idiosyncratic contortions appear in different places across different bootstrap samples, and get voted away by the majority.

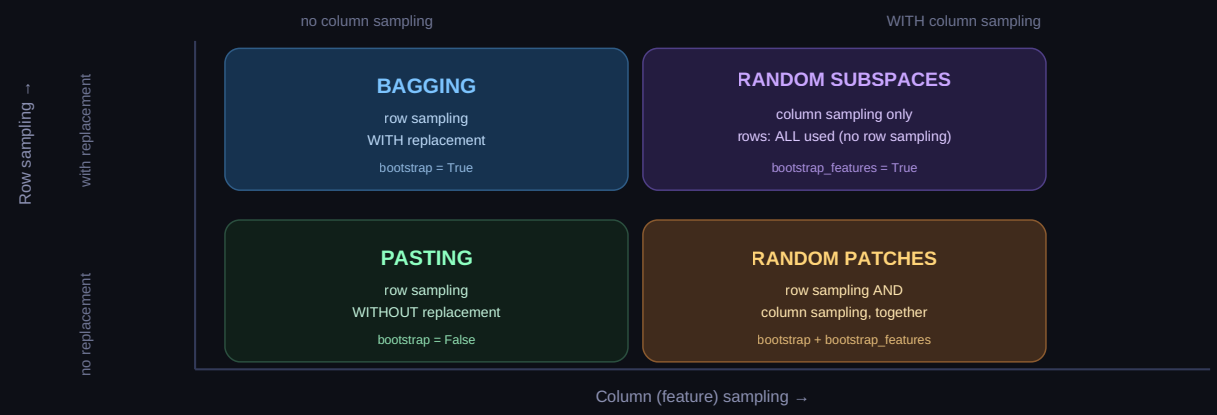
KEY IDEA · RULE OF THUMB

If your single base model's decision boundary looks "jagged" or "spiky," that model has high variance and is a great candidate for Bagging. If the boundary is already too smooth or too simple (a straight line trying to separate a spiral, for example), Bagging won't help much — that's a high-*bias* problem, and no amount of averaging fixes bias.

SECTION 08

The Four Flavors of Bagging

"Bagging" is actually a family of techniques. What differs between them is **how** each base model's subset of data is created — you can sample rows, columns, both, or neither the same way twice.



The four bagging variants, organised by whether rows are sampled with or without replacement, and whether columns (features) are sampled at all.

Technique	Row Sampling	Column Sampling	scikit-learn Setting	Best For
Bagging	With replacement	None (all columns)	`bootstrap=True`	Default choice, general-purpose, most datasets
Pasting	Without replacement	None (all columns)	`bootstrap=False`	Very large datasets where duplication isn't needed
Random Subspaces	None (all rows)	Sampled (with or without replacement)	`bootstrap=False, bootstrap_features=True`	High-dimensional data (many columns)
Random Patches	Sampled	Sampled	`bootstrap=True, bootstrap_features=True`	Very high-dimensional, very large datasets

WARNING · IMPORTANT CAVEAT

Column sampling (Random Subspaces / Random Patches) only helps when you actually have a large number of features — think 50, 100, 200+ columns. If your dataset only has 2 or 3 features and you sample columns down to just 1, you're throwing away most of the signal, and accuracy will drop, not improve. On low-dimensional data, stick to plain Bagging or Pasting.

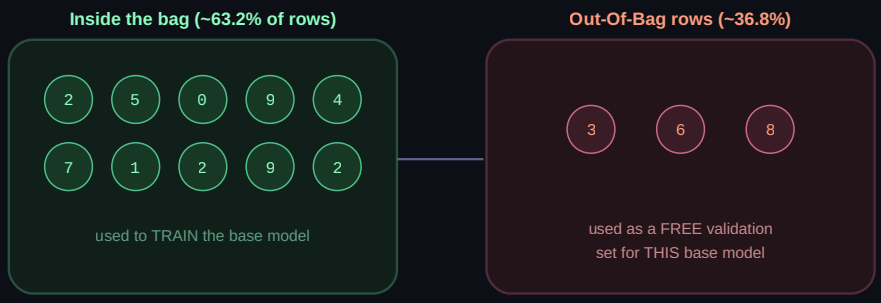
Bagging vs. Pasting — which wins?

In practice, plain **Bagging** (with replacement) tends to outperform **Pasting** (without replacement) on most datasets. The extra randomness from replacement creates more diverse bootstrap samples, which pushes variance down further — slightly hurting bias, but the net effect on new/unseen data is usually positive. Still, both are worth trying and comparing via cross-validation on your specific dataset.

SECTION 09

Out-of-Bag (OOB) Score

Remember the ~36.8% of rows that never make it into a given bootstrap sample (Section 03)? They're not wasted — they hand us a free validation set.



*For any single base model, the rows that **were** sampled (green, left) train it. The rows that were **never** sampled (red, right) — the Out-Of-Bag rows — were never seen by this model, so they can be used to test it, for free.*

How the OOB score is computed

1. For every training row, find all the base models that **did not** include that row in their bootstrap sample.
2. Ask only those models to predict that row (majority vote among just them).
3. Compare that prediction against the row's true label.
4. Average this across every row in the training set → the **OOB score**.

MATH · WHERE THE 36.8% NUMBER COMES FROM, AGAIN

From Section 03: P(a row is left out of one bootstrap sample) = $(1 - 1/N)^N \to 1/e \approx$ **0.368** as N grows. So on average, roughly **36.8% of rows** are "out of bag" for any given base model, and **63.2%** are "in bag." This isn't an approximation pulled from experiments — it is a direct consequence of sampling with replacement.

The huge advantage: you get a reliable estimate of test-set performance **without setting aside a separate validation split**. Every single row contributes to training *and* to validation (just never for the same model).

CODE · IN CODE

```python
from sklearn.ensemble import BaggingClassifier

bag = BaggingClassifier(
    n_estimators=500,
    oob_score=True,   # turn on OOB evaluation
    bootstrap=True,   # must be True for OOB to make sense
    random_state=42
)
bag.fit(X_train, y_train)

print(bag.oob_score_)  # e.g. 0.9214 — a free estimate of test accuracy
```

In practice, the OOB score tends to land close to (though not exactly equal to) the score you'd get from a proper held-out test set — it's a very convenient sanity check while you're still experimenting.

SECTION 10

Key Hyperparameters, in One Table

Every knob scikit-learn's `BaggingClassifier` / `BaggingRegressor` exposes, what it controls, and where to start.

Parameter	What it Controls	Typical Starting Value
`estimator`	The base algorithm each ensemble member uses (defaults to a Decision Tree)	`DecisionTreeClassifier()`
`n_estimators`	How many base models to train	100 – 500 (see Section 11 for the tradeoff)
`max_samples`	Rows given to each base model (fraction or count)	0.25 – 0.5 of training rows
`max_features`	Columns given to each base model (fraction or count)	1.0 unless you have many features
`bootstrap`	Sample rows with replacement? True = Bagging, False = Pasting	`True`
`bootstrap_features`	Sample columns with replacement?	`False` unless doing Random Subspaces/Patches
`oob_score`	Compute the free Out-of-Bag validation score	`True` (costs little, very useful)
`n_jobs`	Parallelise training across CPU cores	`-1` (use all cores)
`random_state`	Reproducibility of the random sampling	Any fixed integer

KEY IDEA · THE TWO LEVERS THAT MATTER MOST

`n_estimators` and `max_samples` move the needle the most in practice. Base-estimator choice matters too (see below), but these two are where to focus your first round of tuning.

Which base algorithm should you bag?

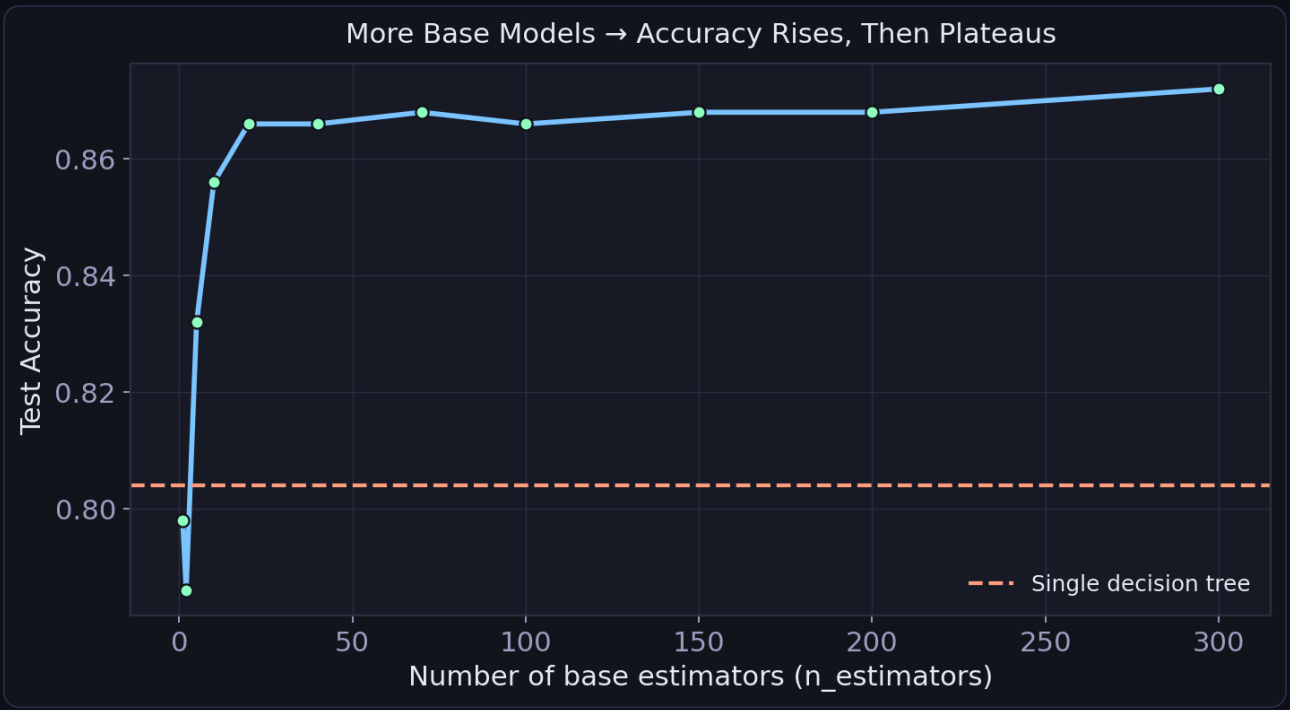
Bagging shines when the base model has **high variance**. That's why decision trees (especially deep, unpruned ones) are the overwhelmingly popular choice.

Base Model	Effect of Bagging
Decision Tree (deep)	Large improvement — this is the classic, textbook use case
KNN	Mild improvement — KNN already has moderate variance
SVM	Often little to no improvement, sometimes slightly worse — SVMs already tend to have low variance, so there's less to fix, and bagging just adds training time
Linear / Logistic Regression	Little benefit — these are naturally high-bias, low-variance models; bagging targets the wrong problem here

SECTION 11

How Many Trees Do You Need?

More base models means more variance reduction — but as we saw in the math of Section 06, there's a hard floor set by the correlation between models. Past a certain point, adding more trees mostly just costs training time.



Test accuracy of a Bagging ensemble as `n_estimators` increases, on a 2,000-row synthetic classification dataset. Accuracy jumps quickly in the first ~20 trees, then plateaus — while a single decision tree (dashed line) is left far behind.

MATH · READING THE CURVE

The steep initial rise matches the $[(1-\rho)/B]\cdot\sigma^2_m$ term from Section 06 shrinking rapidly for small B. The plateau matches that term becoming negligible, leaving only the irreducible $\rho\cdot\sigma^2_m$ floor. In practice, most of the achievable gain shows up by 50–100 trees; going from 100 to 500 rarely buys much beyond a fraction of a percent.

KEY IDEA · PRACTICAL GUIDANCE

- Start around **100** estimators for quick experimentation.
- For a final/production model, **300–500** is a safe, diminishing-returns zone.
- If training time is a bottleneck, set `n_jobs=-1` to parallelise across CPU cores before reducing tree count.

- There's essentially no risk of overfitting by adding more trees — unlike max depth in a single tree, more base models in Bagging do not increase variance. The only cost is compute time.

SECTION 12

Hands-On: Python From Scratch

Let's go from theory to a full working implementation using scikit-learn, the way you'd actually build this in a notebook.

1. Imports and a synthetic dataset

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.metrics import accuracy_score

X, y = make_classification(n_samples=10000, n_features=10, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

2. Baseline: a single decision tree

```
tree = DecisionTreeClassifier(random_state=42)
tree.fit(X_train, y_train)
y_pred = tree.predict(X_test)
print("Single Tree Accuracy:", accuracy_score(y_test, y_pred))
# Single Tree Accuracy: ~0.80
```

3. Bagging on top of the same base model

```
bag = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=50,
    max_samples=0.25,      # each tree sees 25% of training rows
    bootstrap=True,          # sampling WITH replacement
    n_jobs=-1,               # use all CPU cores
    random_state=42
)
bag.fit(X_train, y_train)
y_pred = bag.predict(X_test)
```

```
print("Bagging Accuracy:", accuracy_score(y_test, y_pred))
# Bagging Accuracy: ~0.93  →  a solid jump over the single tree
```

4. Inspect what each base model actually saw

```
# Row indices used by the first base tree
print(bag.estimators_samples_[0].shape)   # (2000,)  → 0.25 * 8000 training rows

# Columns used by the first base tree
print(bag.estimators_features_[0].shape)  # (10,)    → all columns, since max_features=1.0
```

5. Trying Pasting (no replacement)

```
paste = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=50,
    max_samples=0.25,
    bootstrap=False,   # <-- this is the only change: sampling WITHOUT replacement
    n_jobs=-1,
    random_state=42
)
paste.fit(X_train, y_train)
print("Pasting Accuracy:", accuracy_score(y_test, paste.predict(X_test)))
# Usually a touch lower than plain Bagging — less variance reduction
```

6. Random Subspaces (column sampling only)

```
subspace = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=50,
    max_samples=1.0,             # use ALL rows
    bootstrap=False,               # no row sampling
    max_features=0.5,              # each tree only gets 50% of columns
    bootstrap_features=True,
    random_state=42
)
subspace.fit(X_train, y_train)
```

7. Random Patches (row + column sampling together)

```
patches = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
```

```
    n_estimators=50,
    max_samples=0.25,
    bootstrap=True,
    max_features=0.5,
    bootstrap_features=True,
    random_state=42
)
patches.fit(X_train, y_train)
```

8. Turning on the OOB score

```
bag_oob = BaggingClassifier(
    n_estimators=200, bootstrap=True, oob_score=True, random_state=42
)
bag_oob.fit(X_train, y_train)
print("OOB Score:", bag_oob.oob_score_)
print("Test Accuracy:", accuracy_score(y_test, bag_oob.predict(X_test)))
# The two numbers should be close — OOB is a good proxy for test accuracy
```

TRY IT · TRY IT INTERACTIVELY

Before wiring up code, it helps enormously to build (or use) a small interactive demo — a 2D toy dataset with sliders for `n_estimators`, `max_samples`, `bootstrap`, and `max_features` — and watch the decision boundary reshape itself in real time as you change each one. Tools like Streamlit make this a 30-minute build and are extremely effective for building intuition before you touch a real project.

SECTION 13

Practical Tuning Tips (From Real Experiments)

These rules of thumb come from running Bagging across many different datasets and comparing results. They won't be true 100% of the time, but they are an excellent starting point before you run your own search.

1. **Bagging usually beats Pasting.**
   Sampling with replacement introduces more variety between base models, which drives variance down further. Still, always verify with cross-validation on your own data — "usually" is not "always."

2. **Sweet spot for `max_samples` : 0.25 to 0.5.**
   Giving each base model roughly a quarter to half of the training rows tends to strike the best balance between per-model accuracy and inter-model diversity. Too small (like 0.05) starves each model of data; too large (close to 1.0) makes all the bootstrap samples too similar to each other, reducing diversity.

3. **Only use column sampling on high-dimensional data.**
   Random Subspaces and Random Patches are built for datasets with many features (dozens to hundreds+). On a dataset with only 2–5 features, column sampling throws away too much signal and usually hurts accuracy.

4. **Always compare against a single base model.**
   Before celebrating a Bagging result, check the accuracy of one plain decision tree on the same split. If Bagging isn't clearly ahead, either your base model didn't have high variance to begin with, or your hyperparameters need adjusting.

5. **Don't skip `oob_score=True` .**
   It's essentially free (uses data you're already generating) and gives you an extra sanity check that doesn't cost you a held-out validation split.

WARNING · DON'T GUESS — MEASURE

These are starting points, not laws. The right values genuinely differ across datasets. The only reliable way to find the best combination for *your* data is a systematic search — which is exactly what Section 14 covers.

SECTION 14

Finding Hyperparameters with GridSearchCV

Instead of manually trying combinations of `n_estimators`, `max_samples`, `bootstrap`, and `max_features`, let scikit-learn systematically search the grid and cross-validate each combination for you.

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import BaggingClassifier

param_grid = {
    'n_estimators': [50, 500],
    'max_samples': [0.25, 0.5, 0.75, 1.0],
    'bootstrap': [True, False],
    'max_features': [0.5, 0.7, 1.0],
}

search = GridSearchCV(
    estimator=BaggingClassifier(random_state=42),
    param_grid=param_grid,
    cv=5,
    n_jobs=-1,
    scoring='accuracy'
)
search.fit(X_train, y_train)

print("Best params:", search.best_params_)
print("Best CV accuracy:", search.best_score_)
# Example result on a real run:
# Best params: {'bootstrap': True, 'max_features': 1.0, 'max_samples': 0.25, 'n_estimators': 500
# Best CV accuracy: ~0.98
```

KEY IDEA · READING A TYPICAL RESULT

When a grid search comes back recommending `bootstrap=True`, it's effectively confirming Section 13's rule of thumb — plain Bagging outperformed Pasting on that dataset. A recommendation of `max_samples=0.25` confirms the "start low" guidance too. Use grid search results like this to sanity-check your intuition, not replace it.

WARNING · GRID SEARCH IS SLOW

Every combination trains `n_estimators` models, `cv` times. A grid with 2×4×2×3 = 48 combinations at 5-fold CV means **240 full Bagging fits**. For large grids or datasets, prefer

`RandomizedSearchCV`, which samples a fixed number of combinations instead of testing every single one — usually finding a near-optimal result in a fraction of the time.

SECTION 15 · ADVANCED

From Bagging to Random Forest

If everything up to this point made sense, you're one short conceptual step away from understanding one of the most popular algorithms in all of machine learning.

Random Forest is, at its core, *Bagging where the base estimator is always a Decision Tree, plus one extra trick*: at every single split inside every tree, only a random subset of features is even considered as a candidate for splitting (not just once per tree, like Random Subspaces — but freshly re-randomised at every node).

Bagging (general)	**Any base model · row sampling · optional one-time column sampling per model**
Random Forest	**Always Decision Trees · row sampling · fresh random column subset at every split**

Why does this extra randomness help? Recall the variance formula from Section 06:

$$\text{Var(average)} = \rho \cdot \sigma^2_m + [(1-\rho)/B] \cdot \sigma^2_m$$

THE CORRELATION FLOOR, REVISITED

Ordinary Bagging can't push ρ (correlation between trees) down much further once row sampling has done its job — many trees still tend to pick the same "obviously strong" feature at the top of the tree, making them more similar than we'd like. By forcing each split to ignore most features, Random Forest actively decorrelates the trees, pushing ρ lower, which lowers that unavoidable floor in the variance formula even further than plain Bagging can.

KEY IDEA · THE ONE-SENTENCE SUMMARY

Random Forest = Bagging + Decision Trees + column-randomisation at every split. Every concept you learned in this guide — bootstrap sampling, majority voting, OOB score, the bias-variance math, `n_estimators` — carries over directly.

Beyond Random Forest, the same "train many models on different data and combine them" philosophy shows up throughout machine learning — it's the ensemble family's founding idea, with Boosting (train models sequentially, each correcting the last one's mistakes) as the other major branch worth exploring next.

SECTION 16

Summary Cheat-Sheet

Everything from this guide, compressed onto one page.

KEY IDEA · THE CORE IDEA

Train many base models (usually decision trees) on different **bootstrap samples** (random rows, with replacement) of the training data, then combine their predictions by **majority vote** (classification) or **averaging** (regression).

Concept	One-line Definition
Bootstrap sample	Random rows drawn with replacement, usually same size as original data
Base model / estimator	One model trained on one bootstrap sample (commonly a decision tree)
Aggregation	Majority vote (classification) or average (regression) across all base models
Bagging	Row sampling WITH replacement
Pasting	Row sampling WITHOUT replacement
Random Subspaces	Column sampling only, all rows used
Random Patches	Row AND column sampling together
Out-of-Bag (OOB) score	Free validation using the ~36.8% of rows left out of each bootstrap sample
1/e ≈ 0.368	Probability a given row is excluded from a bootstrap sample, as $N \to \infty$
Bias–Variance tradeoff	Bagging reduces variance, leaves bias roughly unchanged
Best base models to bag	High-variance ones: deep decision trees, KNN
Random Forest	Bagging + trees + per-split feature randomisation

Quick-start code block

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

model = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=200,
```

```
    max_samples=0.5,
    bootstrap=True,
    oob_score=True,
    n_jobs=-1,
    random_state=42
)
model.fit(X_train, y_train)
print(model.oob_score_)
```

End of guide · Bagging (Bootstrap Aggregating) · Basics to Advanced